

ОПЕРАЦИОННЫЕ СИСТЕМЫ

ЛАБОРАТОРНЫЙ ПРАКТИКУМ

ЛАБОРАТОРНАЯ РАБОТА № 3

Архитектура памяти Windows

Цель работы:

получение практических навыков по использованию Win32 API для исследования памяти ОС Windows

Методические указания к выполнению лабораторной работы

Типы памяти: физическая память, виртуальная память, память файла подкачки. Совместно используемая физическая память. О физической памяти говорят, что она совместно используется, если она отображается на виртуальное адресное пространство нескольких процессов, хотя виртуальные адреса в каждом процессе могут отличаться. Одно из преимуществ файлов, отображаемых в память, заключается в том, что их легко использовать совместно. Присвоение имени объекту "отображение файла" делает возможным совместное использование файла несколькими процессами. В этом случае его содержимое отображено на совместно используемую физическую память. Возможно также совместное пользование содержимого файла подкачки с помощью механизма отображения файла. Адресное пространство процесса Каждый процесс Win32 получает виртуальное адресное пространство, называемое также адресным пространством, или пространством процесса. Таким образом, код процесса может ссылаться на адреса с &H00000000 по &HFFFFFFF (или с 0 по $2^{32} - 1 = 4\,294\,967\,295$ в десятичной системе счисления). Распределение виртуальной памяти Каждая страница виртуального адресного пространства может находиться в одном из трех состояний: • Reserved (зарезервирована) – страница зарезервирована для использования; • Committed (передана) – для данной виртуальной страницы выделена физическая память в файле подкачки или в файле, отображаемом в память; • Free (свободна) – данная страница не зарезервирована и не передана, и поэтому в данный момент она недоступна для процесса. Кучи памяти в 32-разрядной Windows При создании процесса Windows назначает ему кучу по умолчанию, то есть изначально резервирует область виртуальной памяти объемом 1 Мб. Тем не менее, при необходимости система будет регулировать размер кучи, которая используется самой Windows для различных целей.

Ход выполнения лабораторной работы

КОД ПРОГРАММЫ:

```
#include <iostream>
#include <windows.h>
#include <tlhelp32.h>

void GetMemoryInfo();
void MapProcesses();
void MapVirtualMemory();

void GetMemoryInfo() {
```

```

MEMORYSTATUS memoryStatus;
GlobalMemoryStatus(&memoryStatus);

std::cout << "Memory Status:\n";
std::cout << "  Total Physical Memory: " << memoryStatus.dwTotalPhys << "
bytes\n";
std::cout << "  Available Physical Memory: " << memoryStatus.dwAvailPhys << "
bytes\n";
std::cout << "  Total Virtual Memory: " << memoryStatus.dwTotalVirtual << "
bytes\n";
std::cout << "  Available Virtual Memory: " << memoryStatus.dwAvailVirtual <<
" bytes\n";
}

void MapProcesses() {
    HANDLE hProcessSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (hProcessSnap == INVALID_HANDLE_VALUE) {
        std::cerr << "Error: CreateToolhelp32Snapshot failed\n";
        return;
    }

    PROCESSENTRY32 pe32;
    pe32.dwSize = sizeof(PROCESSENTRY32);
    if (!Process32First(hProcessSnap, &pe32)) {
        std::cerr << "Error: Process32First failed\n";
        CloseHandle(hProcessSnap);
        return;
    }

    std::cout << "Process List:\n";
    do {
        std::cout << "  Process ID: " << pe32.th32ProcessID << ", Parent ID: " <<
pe32.th32ParentProcessID
        << ", Name: " << pe32.szExeFile << '\n';
    } while (Process32Next(hProcessSnap, &pe32));

    CloseHandle(hProcessSnap);
}

void MapVirtualMemory() {
    SYSTEM_INFO sysInfo;
    GetSystemInfo(&sysInfo);
    LPVOID minAddr = sysInfo.lpMinimumApplicationAddress;
    LPVOID maxAddr = sysInfo.lpMaximumApplicationAddress;

    MEMORY_BASIC_INFORMATION memInfo;
    while (VirtualQuery(minAddr, &memInfo, sizeof(memInfo))) {
        std::cout << "Base Address: " << memInfo.BaseAddress << ", Allocation
Base: " << memInfo.AllocationBase
        << ", Region Size: " << memInfo.RegionSize << ", State: " <<
memInfo.State

```

